

9. 索引

1. 本节目标

- 了解什么是索引
- 了解索引使用的数据结构
- 掌握B+树在索引中的应用
- 掌握索引分类和使用

2. 简介

2.1 索引是什么？

MySQL的索引是一种数据结构，它可以帮助数据库高效地查询、更新数据表中的数据。索引通过一定的规则排列数据表中的记录，使得对表的查询可以通过对索引的搜索来加快速度。

MySQL 索引类似于书籍的目录，通过指向数据行的位置，可以快速定位和访问表中的数据，比如汉语字典的目录（索引）页，我们可以按笔画、偏旁部首、拼音等排序的目录（索引）快速查找到需要的字。

- 笔画索引

(三) 难检字笔画索引

(字右边的号码指正文的页码;
带圆括号的字是繁体字或异体字。)

一画	丈	634	卫	517	不	39	为	515	可	268
○	与	605	孑	258	冇	334		518		269
乙		607	也	581	牙	567	尹	594	丙	34
二画		608	飞	129	屯	506	尺	55	左	677
丁	万	350	习	531		665		61	丕	379
		512	乡	540	互	195	夊	168	右	604
七	上	441	四画	(母)	422	(弔)	102	布		39
乂		442	丰	134	中	653	丑	65	戊	526
匕	千	399	亅	394		654	巴	7	平	387
九	乞	396	开	262	内	360	以	586	东	106
刁	川	69	井	247	午	525	予	605	(戊)	616
了	久	251	天	492	壬	424		608	卡	262
	么	329	夫	137	升	449	书	459		399
乃		335		137	夭	577	五画	北		19
也		577	元	612	长	51	未	518	凸	502
	丸	510	无	524		633	末	350	归	172
三画	及	214	云	617	反	126	击	212	且	252
三	亡	512	专	662	爻	578	戈	225		408
干		524	丐	146	乏	123	正	643	甲	223
	丫	566	廿	363	氏	455		644	申	446
予	义	588	五	525		646	甘	147	电	100
于	之	645	(币)	619	丹	86	世	455	由	602
亏	已	586	支	645	鸟	523	本	21	史	454
才	巳	468	丐	343		526	术	462	央	575
下	子	238	卅	432	卞	29		658	(吕)	586

(二) 检字表

(字右边的号码指正文的页码;带圆括号的字是繁体字或异体字。)

1 一部		496	不	39	右	583	而	117	来	275
	上	429	不	111	布	39	死	454	7画	
		429	有	326	平	377	夹	140	奉	132
一	563	3画	友	582	东	102		214	武	509
1画	丰	130	牙	547	且	245		215	表	30
二	118	王	屯	491		397	夷	564	忝	478
丁	100			641	(冊)	44	尧	558	(長)	50
	620	斤	互	190	丘	402	至	626		611
七	382	开	丑	63	丛	73	丞	57	(亞)	549
2画	井	241	4画		册	44	6画		其	384
三	421	天	未	502	丝	453	(所)	463	(來)	275
干	142	夫	末	341	5画		严	551	(東)	102
	145		击	205	考	259	巫	507	画	193
予	65	元	戈	218	老	280	求	403	事	443
于	584	无	正	620	共	155	甫	137	(兩)	294
亏	270	韦		621	亚	549	更	152	枣	601
才	40	云	廿	143	亘	152		153	(面)	335
下	518	专	世	442	吏	289	束	449	(並)	34
丈	611	丐	本	21	冉	598	两	294	亟	208
兀	509	廿	可	261	(互)	152	丽	284		387
与	584	五		261	在	598		289	8画	
	586	(币)	丙	33	百	11	(夾)	140	奏	649
	587	丐	左	652	有	582		214	毒	106
万	341	卅	丕	369		583		215	甚	434

• 拼音索引

汉语拼音音节索引 1

A		cao 操 43	cun 村 79	en 恩 121
a 啊 1		ce 策 44	cuo 搓 80	en 鞞 121
ai 哀 2		cen 岑 45	D	er 儿 121
an 安 3		ceng 层 45	da 搭 81	F
ang 肮 5		cha 插 45	dai 呆 83	Fa 发 123
ao 熬 5		chai 拆 48	dan 单 86	fan 帆 124
B		chan 搀 48	dang 当 88	fang 方 127
ba 八 7		chan 昌 51	dao 刀 90	fei 非 129
bai 白 10		chao 超 53	de 德 92	fen 分 132
ban 班 12		che 车 54	dei 得 94	feng 风 134
bang 帮 14		chen 尘 55	den 钝 94	fo 佛 136
bao 包 15		chen 称 57	deng 登 94	fou 否 136
bei 杯 18		chi 吃 59	di 低 95	fu 夫 137
ben 奔 20		chon 充 62	dia 嗲 99	G
beng 崩 22		chou 抽 64	dian 颠 99	Ga 嘎 144
bi 逼 23		chu 初 65	diao 刁 102	gai 该 145
bian 边 27		chua 欸 68	die 爹 103	gan 干 146
biao 标 30		chuai 揣 68	ding 丁 104	gang 钢 149
bie 别 32		chuan 川 69	diu 丢 106	gao 高 151
bi 宾 32		chuang 窗 70	dong 东 106	ge 哥 152
bing 兵 33		chui 吹 71	dou 兜 108	gei 给 155
bo 玻 35		chun 春 71	du 都 109	gen 根 156
bu 不 38		chuo 戳 72	duan 端 112	geng 耕 156
C		ci 词 73	dui 堆 113	gong 工 158
Ca 擦 40		cong 聪 75	dun 吨 114	gou 沟 160
cai 猜 41		cou 凑 76	duo 多 116	gu 姑 162
can 餐 42		cu 粗 76	E	gua 瓜 167
cang 仓 43		cuan 撺 77	e 鹅 118	guai 乖 168
		cui 崔 78	ei 欸 120	guan 关 168

2.2 为什么要使用索引?

显而易见，使用索引的目的只有一个，就是提升数据检索的效率，在应用程序的运行过程中，查询操作的频率远远高于增删改的频率。

3. 索引应该选择哪种数据结构

3.1 HASH

时间复杂度是 $O(1)$ ，查询速度非常快，但是MySQL并没有选择HASH做为索引的默认数据结构，主要原因是HASH不支持范围查找

3.2 二叉搜索树

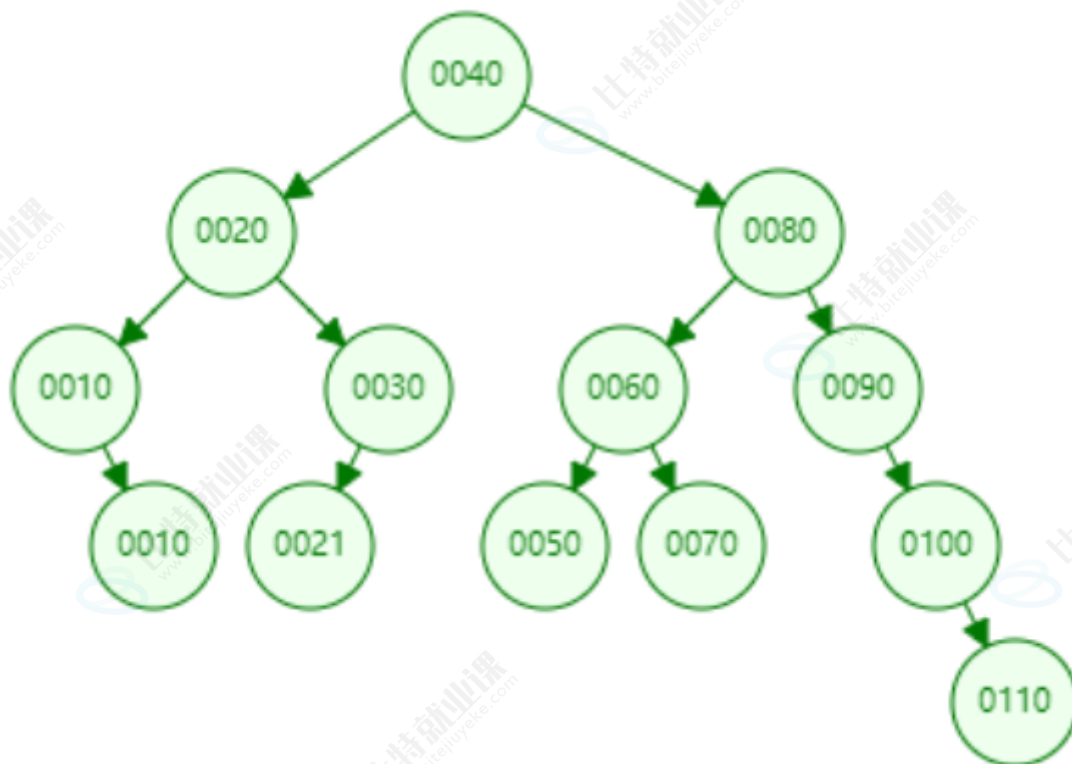
二叉搜索树的中序遍历是一个有序数组，但有几个问题导致它不适合用作索引的数据结构

1. 最坏情况下时间复杂度为 $O(N)$

2. 节点个数过多无法保证树高

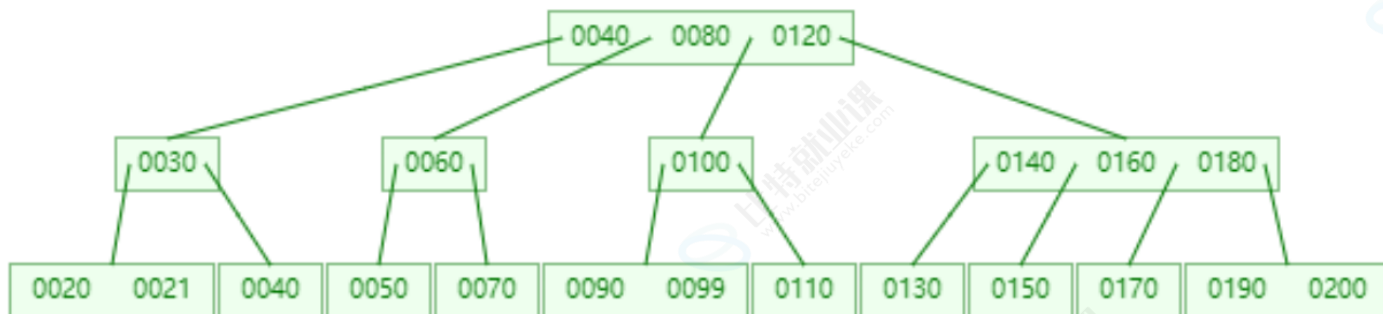
AVL和红黑树，虽然是平衡或者近似平衡，但是毕竟是二叉结构

在检索数据时，每次访问某个节点的子节点时都会发生一次磁盘IO，而在整个数据库系统中，IO是性能的瓶颈，减少IO次数可以有效的提升性能



3.3 N叉树

为了解决树高的问题，可以使用N叉树

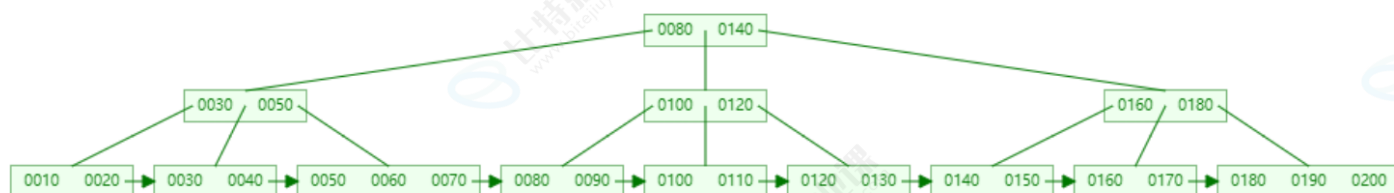


通过观察，相同数据量的情况下，N叉树的树高可以得到有效的控制，也就意味着在相同数据量的情况下可以减少IO的次数，从而提升效率。但是MySQL认为N叉树做为索引的数据结构还不够好。

3.4 B+树

3.4.1 简介

B+树是一种经常用于数据库和文件系统等场合的平衡查找树，MySQL索引采用的数据结构，以4阶B+树为例，如下图所示：



3.4.2 B+树的特点

- 能够保持数据稳定有序，插入与修改有较稳定的时间复杂度
- 非叶子节点仅具有索引作用，不存储数据，所有叶子节点保真实数据
- 所有叶子节点构成一个有序链表，可以按照key排序的次序依次遍历全部数据

3.4.3 B+树与B树的对比

- 叶子节点中的数据是连续的，且相互链接，便于区间查找和搜索。
- 非叶子节点的值都包含在叶子节点中
- 对于B+树而言，在相同树高的情况下，查找任一元素的时间复杂度都一样，性能均衡。

4. MySQL中的页

4.1 为什么要使用页

- 在 `.ibd` 文件中最重要的结构体就是**Page(页)**，页是内存与磁盘交互的最小单元，默认大小为**16KB**，每次内存与磁盘的交互至少读取一页，所以在磁盘中每个页内部的地址都是连续的，之所以这样做，是因为在使用数据的过程中，根据局部性原理，将来要使用的数据大概率与当前访问的数据在空间上是临近的，所以一次从磁盘中读取一页的数据放入内存中，当下次查询的数据还在这个页中时就可以从内存中直接读取，从而减少磁盘I/O提高性能

局部性原理：

是指程序在执行时呈现出局部性规律，在一段时间内，整个程序的执行仅限于程序中的某一部分。相应地，执行所访问的存储空间也局限于某个内存区域，局部性通常有两种形式：时间局部性和空间局部性。

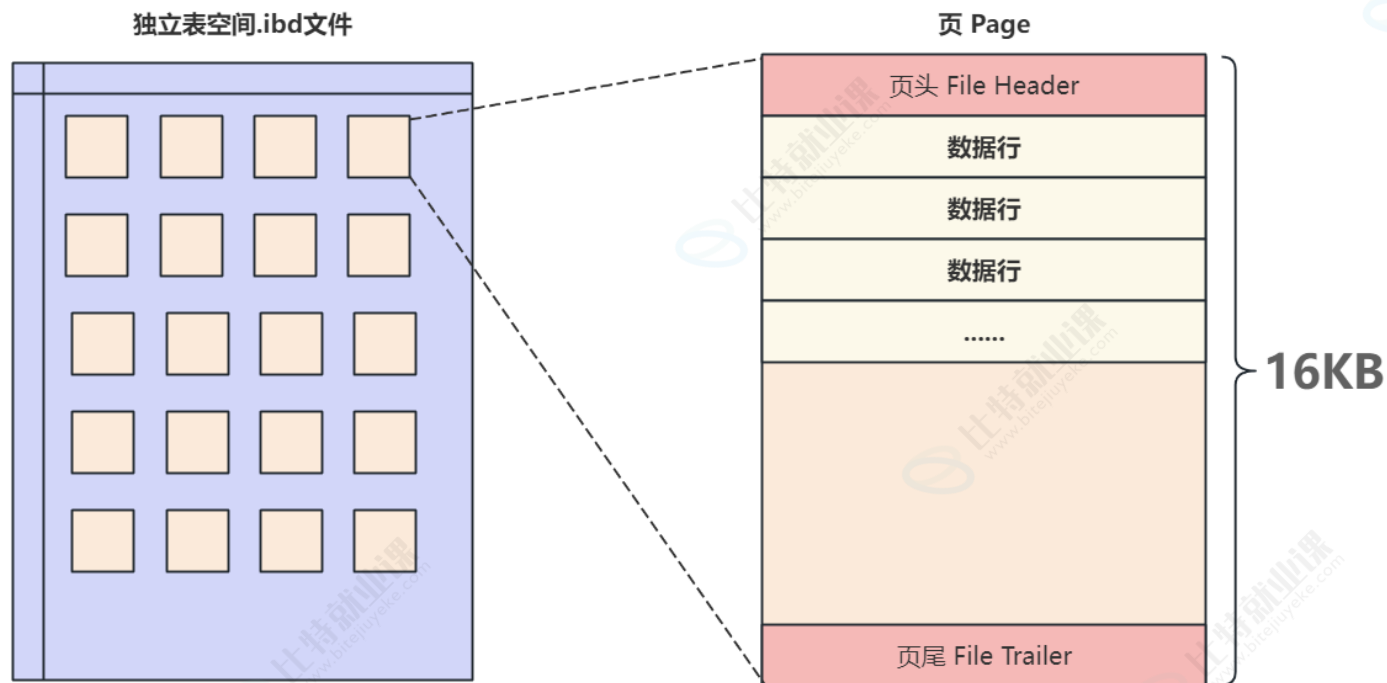
时间局部性 (Temporal Locality)：如果一个信息项正在被访问，那么在近期它很可能还会被再次访问。

空间局部性 (Spatial Locality)：将来要用到的信息大概率与正在使用的信息在空间地址上是临近的。

- 每一个页中即使没有数据也会使用 **16KB** 的存储空间，同时与索引的B+树中的节点对应，后续在**专题七：索引**中详细讲解B+树的内容，查看页的大小，可以通过系统变量 `innodb_page_size` 查看

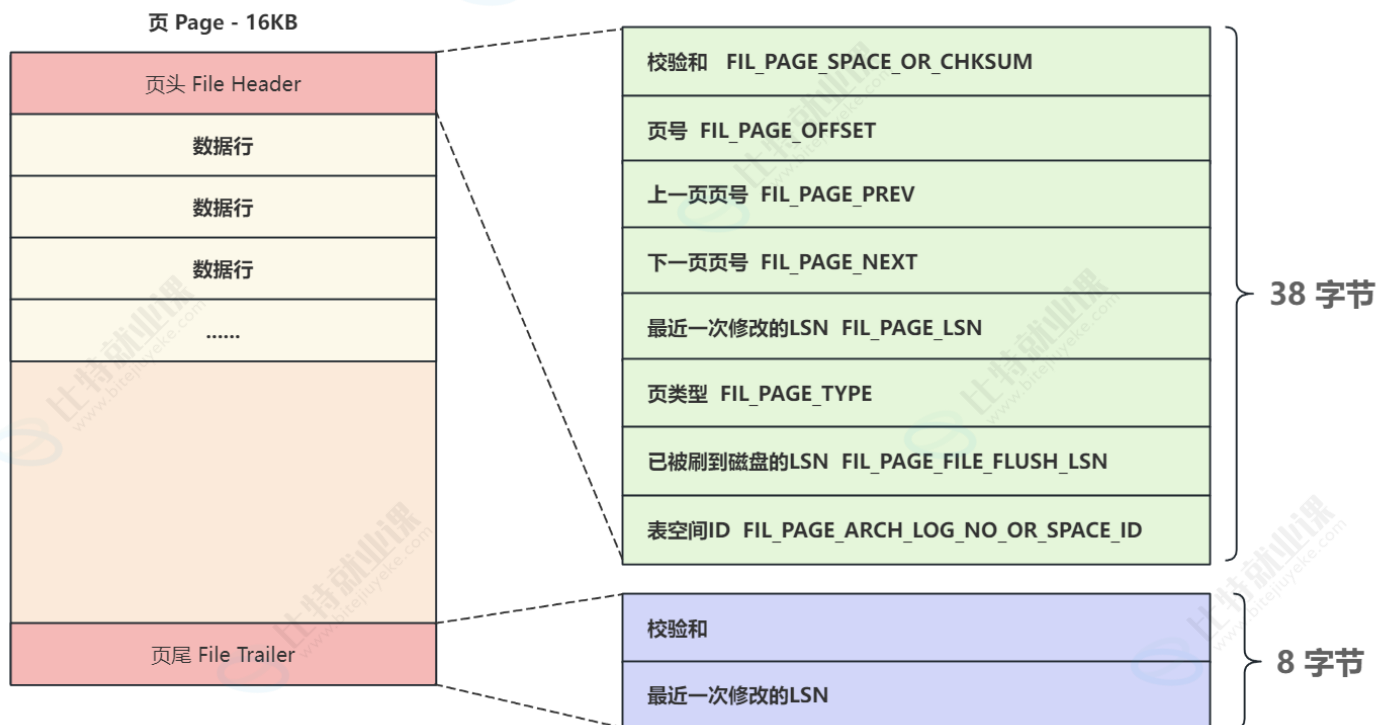
```
1 mysql> SHOW VARIABLES LIKE 'innodb_page_size';
2 +-----+-----+
3 | Variable_name | Value |
4 +-----+-----+
5 | innodb_page_size | 16384 | # 16KB
6 +-----+-----+
7 1 row in set, 1 warning (0.04 sec)
```

- 在MySQL中有多种不同类型的页，最常用的就是用来存储数据和索引的**"索引页"**，也叫做**"数据页"**，但不论哪种类型的页都会包含**页头(File Header)**和**页尾(File Trailer)**，页的主体信息使用**数据"行"**进行填充，**数据页**的基本结构如下图所示：



4.2 页文件头和页文件尾

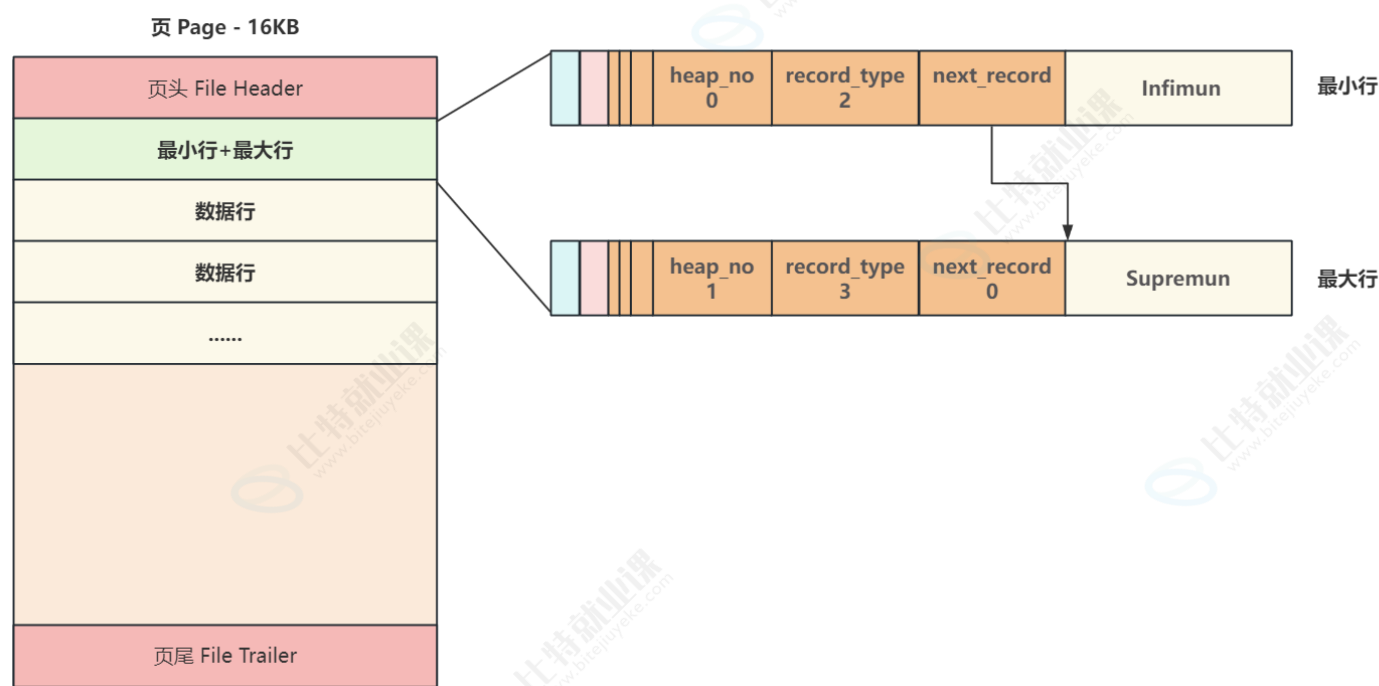
页文件头和页文件尾中包含的信息，如下图所示：



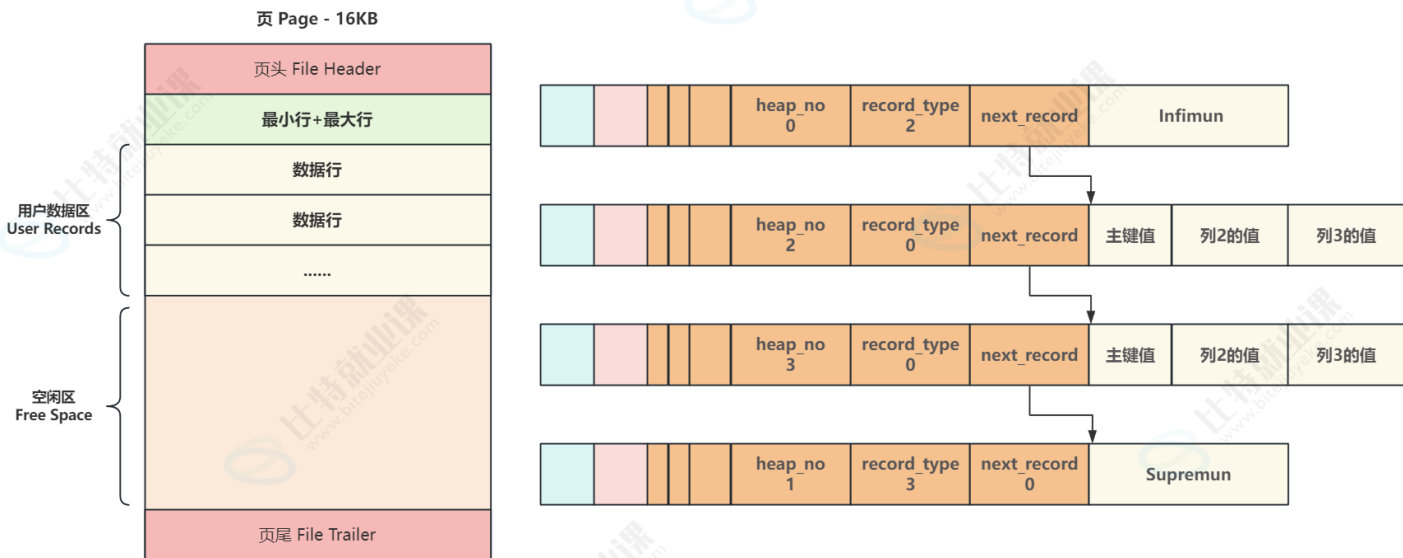
这里我们只关注，**上一页页号**和**下一页页号**，通过这两个属性可以把页与页之间链接起来，形成一个**双向链表**。

4.3 页主体

- 页主体部分是保存真实数据的主要区域，每当创建一个新页，都会自动分配两个行，一个是页内最小行 **Infimun**，另一个是页内最大行 **Supremun**，这两个行并不存储任何真实信息，而是做为数据行链表的头和尾，第一个数据行有一个记录下一行的地址偏移量的区域 **next_record** 将页内所有数据行组成了一个单向链表，此时新页的结构如下所示：



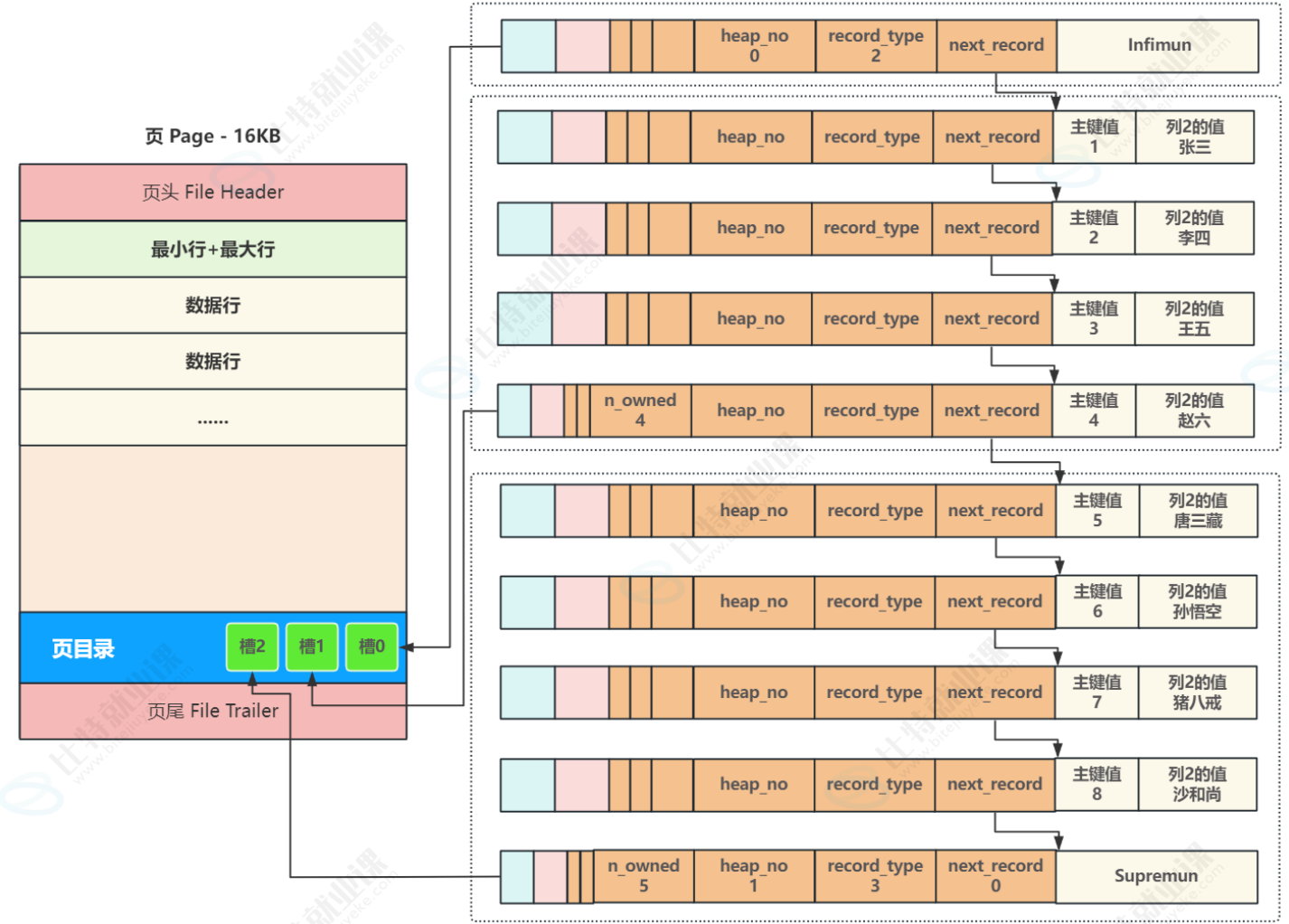
- 当向一个新页插入数据时，将 **Infimun** 连接第一个数据行，最后一行真实数据行连接 **Supremun**，这样数据行就构建成了一个单向链表，更多的行数据插入后，会按照主键从小到大的顺序进行链接，如下图所示



4.4 页目录

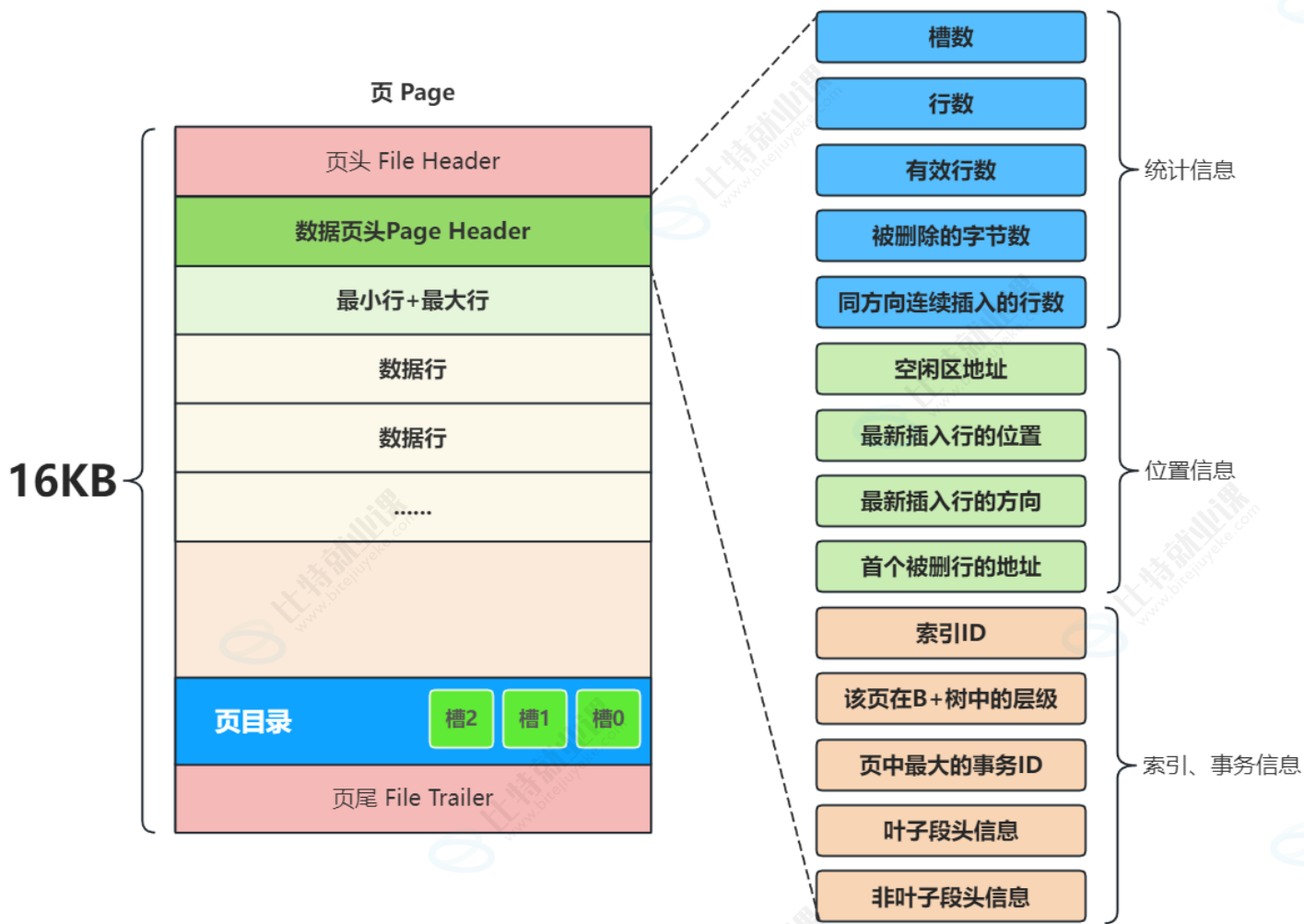
- 当按主键或索引查找某条数据时，最直接简单的方法就是从头行 **infimun** 开始，沿着链表顺序逐个比对查找，但一个页有16KB，通常会存在数百行数据，每次都要遍历数百行，无法满足高效查询，为了提高查询效率，InnoDB采用二分查找来解决查询效率问题；

- 具体实现方式是，在每一个页中加入一个叫做**页目录 Page Directory**的结构，将页内包括头行、尾行在内的所有行进行分组，约定头行单独为一组，其他每个组最多8条数据，同时把每个组最后一行在页中的地址，按主键从小到大的顺序记录在页目录中在，页目录中的每一个位置称为一个**槽**，每个槽都对应了一个分组，一旦分组中的数据行超过分组的上限8个时，就会分裂出一个新的分组；
- 后续在查询某行时，就可以通过二分查找，先找到对应的槽，然后在槽内最多8个数据行中进行遍历即可，从而大幅提高了查询效率，这时一个页的核心结构就完成了；
- 例如要查找主键为6的行，先比对槽中记录的主键值，定位到最后一个槽2，再从最后一个槽中的第一条记录遍历，第二条记录就是我们要查询的目标行。



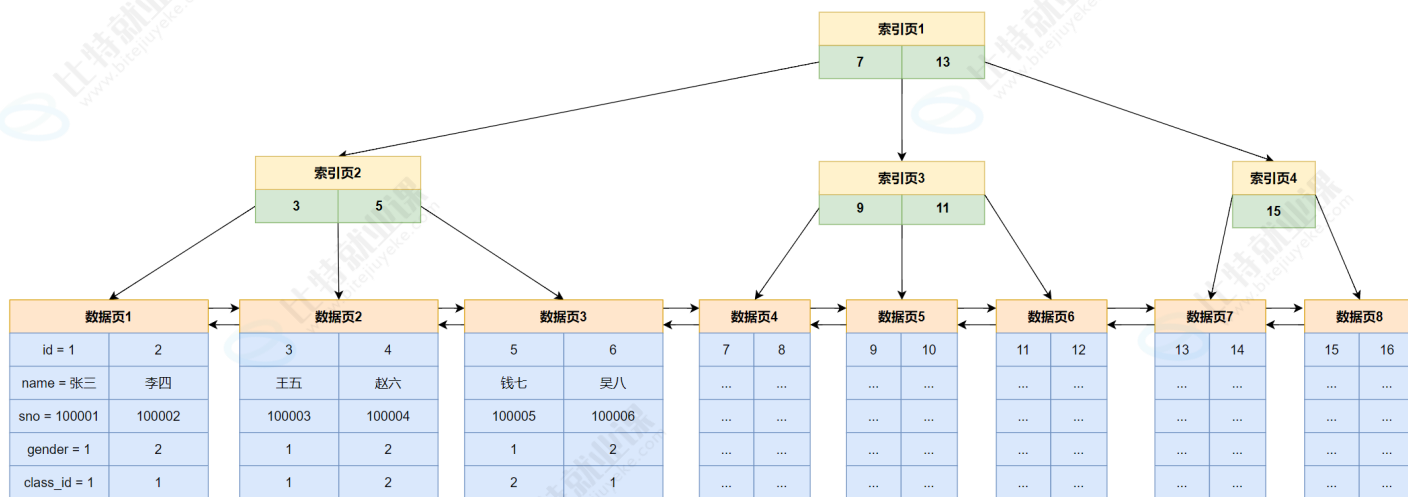
4.5 数据页头

数据页头记录了当前页保存数据相关的信息，如下图所示



5. B+在MySQL索引中的应用

- 非叶子节点保存索引数据，叶子节点保存真实数据，如下图所示



- 以查找id为5的记录，完整的检索过程如下：

- 首先判断B+树的根节点中的索引记录，此时 $5 < 7$ ，应访问左孩子节点，找到索引页2
- 在索引页2中判断id的大小，找到与5相等的记录，命中，加载对应的数据页

以上的IO过程，加载索引页1 --> 加载索引页2 --> 加载数据页3

5.1 计算三层树高的B+树可以存放多少条记录

- 假设一条用户数据大小为1KB，在忽略数据页中数据页自身属性空间占用的情况下，一页可以存16条数据
- 索引页一条数据的大小为，主键用BIGINT类型占8Byte，下一页地址6Byte，一共是14Byte，一个索引页可以保存 $16 \times 1024 / 14 = 1170$ 条索引记录
- 如果只有三层树高的情况，综合只保存索引的根节点和二级节点的索引页以及保存真实数据的数据页，那么一共可以保存 $1170 \times 1170 \times 16 = 21,902,400$ 条记录，也就是说在两千多万条数据的表中，可以通过三次IO就完成数据的检索

6. 索引分类

6.1 主键索引

- 当在一个表上定义一个主键 **PRIMARY KEY** 时，InnoDB使用它作为**聚集索引**。
- 推荐为每个表定义一个主键。如果没有逻辑上唯一且非空的列或列集可以使用主键，则添加一个自增列。

6.2 普通索引

- 最基本的索引类型，没有唯一性的限制。
- 可能为多列创建组合索引，称为复合索引或组全索引

6.3 唯一索引

- 当在一个表上定义一个唯一键 **UNIQUE** 时，自动创建唯一索引。
- 与普通索引类似，但区别在于唯一索引的列不允许有重复值。

6.4 全文索引

- 基于文本列(CHAR、VARCHAR或TEXT列)上创建，以加快对这些列中包含的数据查询和DML操作
- 用于全文搜索，仅MyISAM和InnoDB引擎支持。

6.5 聚集索引

- 与主键索引是同义词
- 如果没有为表定义 **PRIMARY KEY**，InnoDB使用第一个 **UNIQUE** 和 **NOT NULL** 的列作为聚集索引。

- 如果表中没有 **PRIMARY KEY** 或合适的 **UNIQUE** 索引，InnoDB会为新插入的行生成一个行号并用6字节的 **ROW_ID** 字段记录，**ROW_ID** 单调递增，并使用 **ROW_ID** 做为索引。

6.6 非聚集索引

- 聚集索引以外的索引称为非聚集索引或二级索引
- 二级索引中的每条记录都包含该行的主键列，以及二级索引指定的列。
- InnoDB使用这个主键值来搜索聚集索引中的行，这个过程称为回表查询

6.7 索引覆盖

- 当一个select语句使用了普通索引且查询列表中的列刚好是创建普通索引时的所有或部分列，这时就可以直接返回数据，而不用回表查询，这样的现象称为索引覆盖

7. 使用索引

7.1 自动创建

- 当我们为一张表加主键约束(Primary key)，外键约束(Foreign Key)，唯一约束(Unique)时，MySQL会为对应的列自动创建一个索引
- 如果表不指定任何约束时，MySQL会自动为每一列生成一个索引并用 **ROW_ID** 进行标识

7.2 手动创建

7.2.1 主键索引

```
1 # 方式一，创建表时创建主键
2 create table t_test_pk (
3     id bigint primary key auto_increment,
4     name varchar(20)
5 );
6
7 # 方式二，创建表时单独指定主键列
8 create table t_test_pk1 (
9     id bigint auto_increment,
10    name varchar(20),
11    primary key (id)
12 );
13
14 # 方式三，修改表中的列为主键索引
15 create table t_test_pk2 (
16     id bigint,
17     name varchar(20)
```



```
18 );
19
20 alter table t_test_pk2 add primary key (id) ;
21 alter table t_test_pk2 modify id bigint auto_increment;
```

7.2.2 唯一索引

```
1 # 方式一，创建表时创建唯一键
2 create table t_test_uk (
3     id bigint primary key auto_increment,
4     name varchar(20) unique
5 );
6
7 # 方式二，创建表时单独指定唯一列
8 create table t_test_uk1 (
9     id bigint primary key auto_increment,
10    name varchar(20),
11    unique (name)
12 );
13
14 # 方式三，修改表中的列为唯一索引
15 create table t_test_uk2 (
16     id bigint primary key auto_increment,
17     name varchar(20)
18 );
19
20 alter table t_test_uk2 add unique (name) ;
```

7.2.3 普通索引

```
1 # 方式一，创建表时指定索引列
2 create table t_test_index (
3     id bigint primary key auto_increment,
4     name varchar(20) unique
5     sno varchar(10),
6     index(sno)
7 );
8
9 # 方式二，修改表中的列为普通索引
10 create table t_test_index1 (
11     id bigint primary key auto_increment,
12     name varchar(20),
13     sno varchar(10)
```

```
14 );
15 alter table t_test_index1 add index (sno) ;
16
17 # 方式三，单独创建索引并指定索引名
18 create table t_test_index2 (
19     id bigint primary key auto_increment,
20     name varchar(20),
21     sno varchar(10)
22 );
23
24 create index index_name on t_test_index2(sno);
```

7.3 创建复合索引

创建语法与创建普通索引相同，只不过指定多个列，列与列之间用逗号隔开

```
1 # 方式一，创建表时指定索引列
2 create table t_test_index4 (
3     id bigint primary key auto_increment,
4     name varchar(20),
5     sno varchar(10),
6     class_id bigint,
7     index (sno, class_id)
8 );
9
10 # 方式二，修改表中的列为复合索引
11 create table t_test_index5 (
12     id bigint primary key auto_increment,
13     name varchar(20),
14     sno varchar(10),
15     class_id bigint
16 );
17 alter table t_test_index5 add index (sno, class_id);
18
19 # 方式三，单独创建索引并指定索引名
20 create table t_test_index6 (
21     id bigint primary key auto_increment,
22     name varchar(20),
23     sno varchar(10),
24     class_id bigint
25 );
26
27 create index index_name on t_test_index6 (sno, class_id);
```

7.4 查看索引

```
1 # 方式一: show keys from 表名
2 mysql> show keys from t_test_index6\G
3 ***** 1. row *****
4      Table: t_test_index6
5      Non_unique: 0
6      Key_name: PRIMARY
7      Seq_in_index: 1
8      Column_name: id
9      Collation: A
10     Cardinality: 0
11      Sub_part: NULL
12      Packed: NULL
13      Null:
14      Index_type: BTREE
15      Comment:
16 Index_comment:
17      Visible: YES
18      Expression: NULL
19 ***** 2. row *****
20      Table: t_test_index6
21      Non_unique: 1
22      Key_name: index_name
23      Seq_in_index: 1
24      Column_name: sno
25      Collation: A
26     Cardinality: 0
27      Sub_part: NULL
28      Packed: NULL
29      Null: YES
30      Index_type: BTREE
31      Comment:
32 Index_comment:
33      Visible: YES
34      Expression: NULL
35 ***** 3. row *****
36      Table: t_test_index6
37      Non_unique: 1
38      Key_name: index_name
39      Seq_in_index: 2
40      Column_name: class_id
41      Collation: A
42     Cardinality: 0
43      Sub_part: NULL
44      Packed: NULL
```

```

45         Null: YES
46     Index_type: BTREE
47     Comment:
48 Index_comment:
49     Visible: YES
50     Expression: NULL
51 3 rows in set (0.00 sec)
52
53 # 方式二
54 show index from t_test_index6;
55
56 # 方式三, 简要信息: desc 表名;
57 desc t_test_index6;
58

```

7.5 删除索引

7.5.1 主键索引

```

1 # 语法
2 alter table 表名 drop primary key;
3
4 # 示例, 删除t_test_index6表中的主键
5 mysql> alter table t_test_index6 drop primary key;
6 ERROR 1075 (42000): Incorrect table definition; there can be only one auto
   column and it must be defined as a key
7
8 # 如查提示由于自增列的错误, 先删除自增属性
9 mysql> alter table t_test_index6 modify id bigint;
10 Query OK, 0 rows affected (0.04 sec)
11 Records: 0 Duplicates: 0 Warnings: 0
12
13 # 重新删除主键
14 mysql> alter table t_test_index6 drop primary key;
15 Query OK, 0 rows affected (0.04 sec)
16 Records: 0 Duplicates: 0 Warnings: 0
17
18 # 查看结果
19 mysql> show keys from t_test_index6\G
20 ***** 1. row *****
21      Table: t_test_index6
22      Non_unique: 1
23      Key_name: index_name

```

```

24 Seq_in_index: 1
25 Column_name: sno
26 Collation: A
27 Cardinality: 0
28 Sub_part: NULL
29 Packed: NULL
30 Null: YES
31 Index_type: BTREE
32 Comment:
33 Index_comment:
34 Visible: YES
35 Expression: NULL
36 ***** 2. row *****
37 Table: t_test_index6
38 Non_unique: 1
39 Key_name: index_name
40 Seq_in_index: 2
41 Column_name: class_id
42 Collation: A
43 Cardinality: 0
44 Sub_part: NULL
45 Packed: NULL
46 Null: YES
47 Index_type: BTREE
48 Comment:
49 Index_comment:
50 Visible: YES
51 Expression: NULL
52 2 rows in set (0.00 sec)
53

```

7.5.2 其他索引

```

1 # 语法
2 alter table 表名 drop index 索引名;
3
4 # 示例, 删除t_test_index6表中名为index_name的索引
5 mysql> alter table t_test_index6 drop index index_name;
6 Query OK, 0 rows affected (0.02 sec)
7 Records: 0 Duplicates: 0 Warnings: 0
8
9 # 查看结果
10 mysql> show keys from t_test_index6\G
11 Empty set (0.00 sec)

```


7.6 创建索引的注意事项

- 索引应该创建在高频查询的列上
- 索引需要占用额外的存储空间
- 对表进行插入、更新和删除操作时，同时也会修索引，可能会影响性能
- 创建过多或不合理的索引会导致性能下降，需要谨慎选择和规划索引

关于索引的优化与使用，我们在MySQL中级课程中讲解

